Summarize

# Chapter 4. Ambassadors

Chapter 3 introduced the sidecar pattern, where one container augments a pre-existing container to add functionality. This chapter introduces the ambassador pattern, where an ambassador container brokers interactions between the application container and the rest of the world. As with other single-node patterns, the two containers are tightly linked in a symbiotic pairing that is scheduled to a single machine. A canonical diagram of this pattern is shown in Figure 4-1.

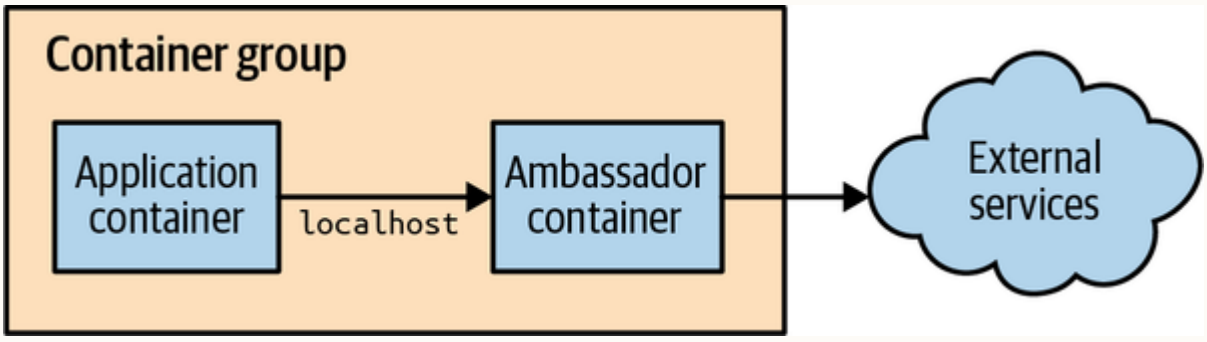



Figure 4-1. Generic ambassador pattern

The value of the ambassador pattern is twofold. First, as with the other single-node patterns, there is inherent value in building modular, reusable containers. The separation of concerns makes the containers easier to build and maintain. Likewise, the ambassador container can be reused with a number of different application containers. This reuse speeds up application development because the container's code can be reused in a number of places. Additionally, the implementation is both more consistent and of a higher quality because it is built once and used in many different contexts.

The rest of this chapter provides a number of examples of using the ambassador pattern to implement a series of real-world applications.

# Using an Ambassador to Shard a Service

Sometimes the data that you want to store in a storage layer becomes too big for a single machine to handle. In such situations, you need to *shard* your storage layer. Sharding splits up the layer into multiple disjoint pieces, each hosted by a separate machine. This chapter focuses on a single-node pattern for adapting an existing service to talk to a sharded service that exists somewhere in the world. It does not discuss how the sharded service came to exist. Sharding and a multinode sharded service design pattern are discussed in great detail in Chapter 7. A diagram of a sharded service is shown in Figure 4-2.

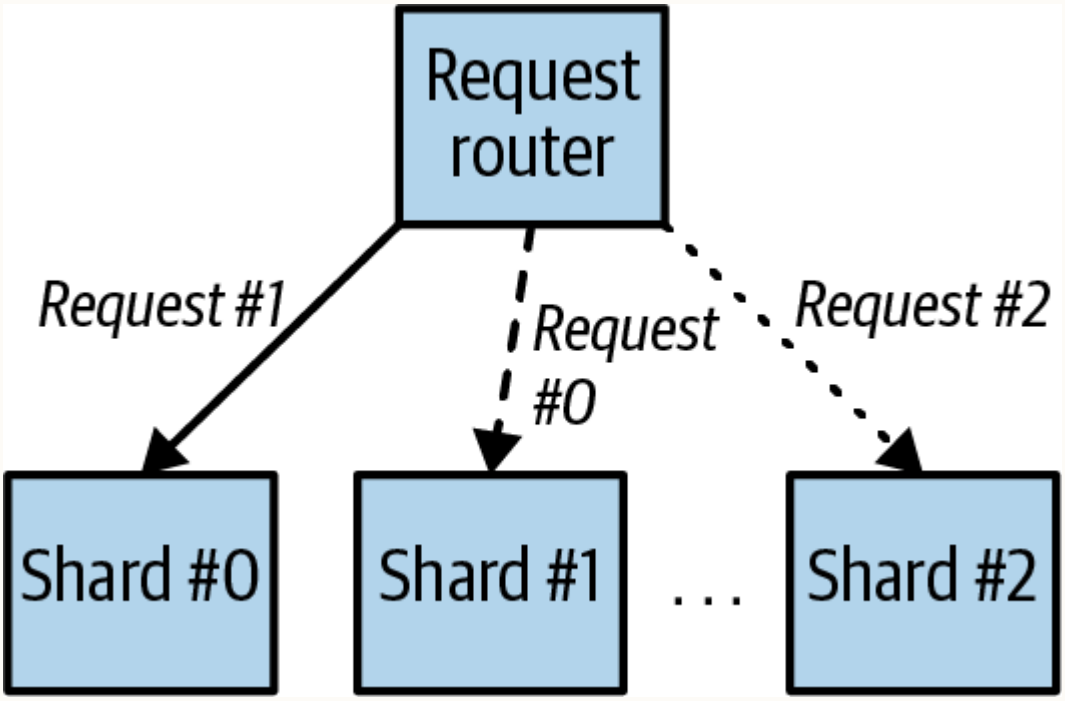



Figure 4-2. A generic sharded service

When deploying a sharded service, one question that arises is how to integrate it with the frontend or middleware code that stores data. Clearly there needs to be logic that routes a particular request to a particular shard, but often it is difficult to retrofit such a sharded client into existing source code that expects to connect to a single storage backend. Additionally, sharded services make it difficult to share configuration between development environments (where there is often only a single storage shard) and production environments (where there are often many storage shards).

One approach is to build all of the sharding logic into the sharded service itself. In this approach, the sharded service also has a stateless load balancer that directs traffic to the appropriate shard. Effectively, this load balancer is a distributed ambassador as a service. This makes a client-side ambassador unnecessary at the expense of a more complicated deployment for the sharded service. The alternative is to integrate a single-node ambas-

sador on the client side to route traffic to the appropriate shard. This makes deploying the client somewhat more complicated but simplifies the deployment of the sharded service. As is always the case with trade-offs, it is up to the particulars of your specific application to determine which approach makes the most sense. Some factors to consider include where team lines fall in your architecture, as well as where you are writing code versus simply deploying off-the-shelf software. Ultimately, either choice is valid. The section "Hands On: Implementing a Sharded Redis" describes how to use the single-node ambassador pattern for client-side sharding.

When adapting an existing application to a sharded backend, you can introduce an ambassador container that contains all of the logic needed to route requests to the appropriate storage shard. Thus, your frontend or middleware application only connects to what appears to be a single storage backend running on localhost. However, this server is in fact actually a *sharding ambassador proxy*, which receives all of the requests from your application code, sends a request to the appropriate storage shard, and then returns the result to your application. This use of an ambassador is diagrammed in Figure 4-3.

The net result of applying the ambassador pattern to sharded services is a separation of concerns between the application container, which simply knows it needs to talk to a storage service and discovers that service on localhost, and the sharding ambassador proxy, which only contains the code necessary to perform appropriate sharding. As with all good single-node patterns, this ambassador can be reused between many different applications. Or, as we'll see in the following hands-on example, an off-the shelf open source implementation can be used for the ambassador, speeding up the development of the overall distributed system.

## Hands On: Implementing a Sharded Redis

Redis is a fast key-value store that can be used as a cache or for more persistent storage. In this example, we'll be using it as a cache. We'll begin by deploying a sharded Redis service to a Kubernetes cluster. We'll use the `StatefulSet` API object to deploy it, since it will give us unique DNS names for each shard that we can use when configuring the proxy.

The `StatefulSet` for Redis looks like this:

```
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
```

```
  name: sharded-redis
spec:
  serviceName: "redis"
  replicas: 3
  template:
    metadata:
      labels:
        app: redis
    spec:
      terminationGracePeriodSeconds: 10
      containers:
      - name: redis
        image: redis
        ports:
        - containerPort: 6379
          name: redis
```

Save this to a file named *redis-shards.yaml*, and you can deploy this with `kubectl create -f redis-shards.yaml`. This will create three containers running redis. You can see these by running `kubectl get pods`; you should see `sharded-redis-[0,1,2]`.

Of course, just running the replicas isn't sufficient; we also need names by which we can refer to the replicas. In this case, we'll use a Kubernetes `Service`, which will create DNS names for the replicas we have created. The `Service` looks like this:

```
apiVersion: v1
kind: Service
metadata:
  name: redis
  labels:
    app: redis
spec:
  ports:
  - port: 6379
    name: redis
  clusterIP: None
  selector:
    app: redis
```

Save this to a file named *redis-service.yaml* and deploy with `kubectl create -f redis-service.yaml`. You should now have DNS entries for `sharded-redis-0.redis`, `sharded-redis-1.redis`, etc. We can use these names to configure `twemproxy`. `twemproxy` is a

lightweight, highly performant proxy for `memcached` and `Redis`, which was originally developed by Twitter and is open source and available on GitHub. We can configure `twemproxy` to point to the replicas we created by using the following configuration:

```
redis:
  listen: 127.0.0.1:6379
  hash: fnv1a_64
  distribution: ketama
  auto_eject_hosts: true
  redis: true
  timeout: 400
  server_retry_timeout: 2000
  server_failure_limit: 1
  servers:
   - sharded-redis-0.redis:6379:1
   - sharded-redis-1.redis:6379:1
   - sharded-redis-2.redis:6379:1
```

In this config, you can see that we are serving the Redis protocol on `localhost:6379` so that the application container can access the ambassador. We will deploy this into our ambassador pod using a Kubernetes `ConfigMap` object that we can create with:

```
kubectl create configmap twem-config --from-file=./nutcracker.yaml
```

Finally, all of the preparations are done and we can deploy our ambassador example. We define a pod that looks like:

```
apiVersion: v1
kind: Pod
metadata:
  name: ambassador-example
spec:
  containers:
    # This is where the application container would go, for example
    # - name: nginx
    #   image: nginx
    # This is the ambassador container
    - name: twemproxy
      image: ganomede/twemproxy
      command:
      - "nutcracker"
      - "-c"
```

```
      - "/etc/config/nutcracker.yaml"
      - "-v"
      - "7"
      - "-s"
      - "6222"
      volumeMounts:
      - name: config-volume
        mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: twem-config
```

This pod defines the ambassador; then the specific user's application container can be injected to complete the pod.

## Using an Ambassador for Service Brokering

When trying to render an application portable across multiple environments (e.g., public cloud, physical data center, or private cloud), one of the primary challenges is service discovery and configuration. To understand what this means, imagine a frontend that relies on a MySQL database to store its data. In the public cloud, this MySQL service might be provided as software as a service (SaaS), whereas in a private cloud it might be necessary to dynamically spin up a new virtual machine or container running MySQL.

Consequently, building a portable application requires that the application know how to introspect its environment and find the appropriate MySQL service to connect to. This process is called *service discovery*, and the system that performs this discovery and linking is commonly called a *service broker*. As with previous examples, the ambassador pattern enables a system to separate the logic of the application container from the logic of the service broker ambassador. The application simply always connects to an instance of the service (e.g., MySQL) running on localhost. It is the responsibility of the service broker ambassador to introspect its environment and broker the appropriate connection. This process is shown in Figure 4-3.

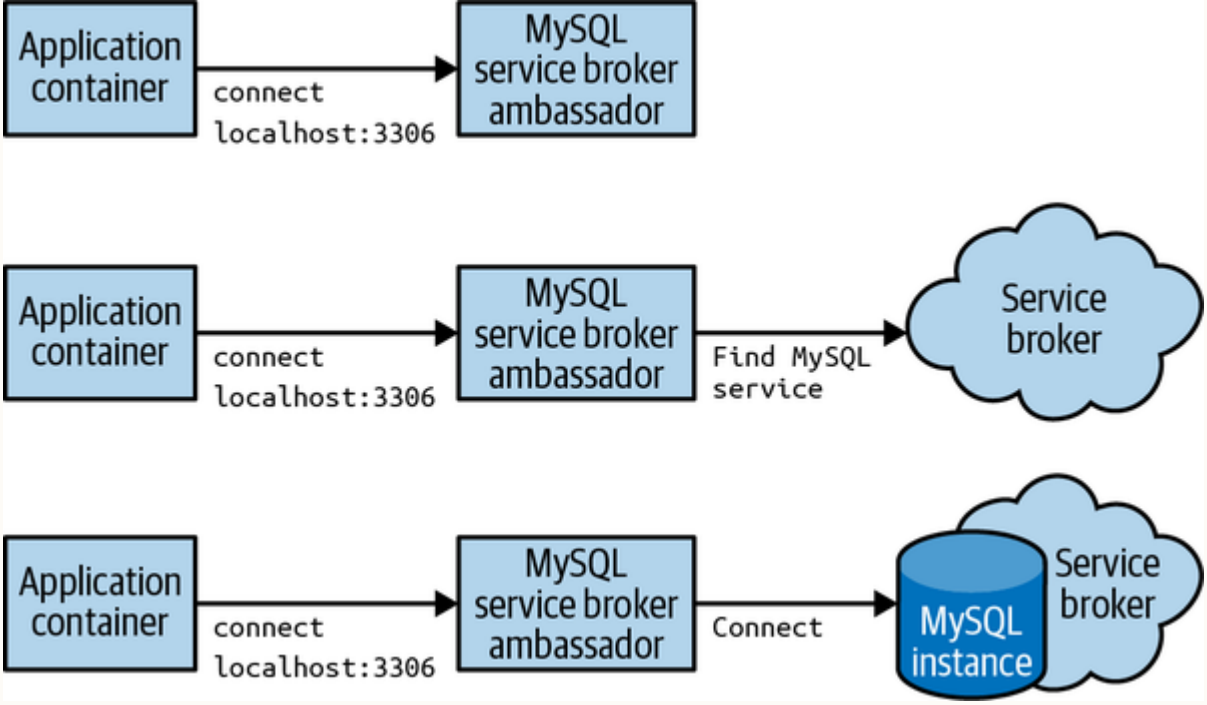



Figure 4-3. A service broker ambassador creating a MySQL service

# Using an Ambassador to Do Experimentation or Request Splitting

A final example application of the ambassador pattern is to perform experimentation or other forms of request splitting. In many production systems, it is advantageous to be able to perform request splitting, where some fraction of all requests are not serviced by the main production service but rather are redirected to a different implementation of the service. Most often, this is used to perform experiments with new beta versions of the service to determine if the new version of the software is reliable or comparable in performance to the currently deployed version.

Additionally, request splitting is sometimes used to `tee` or split traffic such that all traffic goes to both the production system as well as a newer, undeployed version. The responses from the production system are returned to the user, while the responses from the `tee-d` service are ignored. Most often, this form of request splitting is used to simulate production load on the new version of the service without risking impact to existing production users.

Given the previous examples, it is straightforward to see how a request-splitting ambassador can interact with an application container to implement request splitting. As before, the application container simply connects to the service on localhost, while the ambassador container receives the requests, proxies responses to both the production and experimental systems, and then returns the production responses back as if it had performed the work itself.

This separation of concerns keeps the code in each container slim and focused, and the modular factoring of the application ensures that the request-splitting ambassador can be reused for a variety of different applications and settings.

## Hands On: Implementing 10% Experiments

To implement our request-splitting experiment, we're going to use the `nginx` web server. `nginx` is a powerful, richly featured open source server. To configure `nginx` as the ambassador, we'll use the following configuration (note that this is for HTTP but it could easily be adapted for HTTPS as well):

```
worker_processes  5;
error_log  error.log;
pid        nginx.pid;
worker_rlimit_nofile 8192;

events {
  worker_connections  1024;
}

http {
    upstream backend {
        ip_hash;
        server web weight=9;
        server experiment;
    }

    server {
        listen localhost:80;
        location / {
            proxy_pass http://backend;
        }
    }
}
```

NOTE

As with the previous discussion of sharded services, it's also possible to deploy the experiment framework as a separate microservice in front of your application instead of integrating it as a part of your client pods. Of course, by doing this you are introducing another service that needs to be maintained, scaled, monitored, etc. If experimentation is likely to be a longstanding component in your architecture, this might be worthwhile. If it is used more occasionally, then a client-side ambassador might make more sense.

---

You'll note that I'm using IP hashing in this configuration. This is important because it ensures that the user doesn't flip-flop back and forth between the experiment and the main site. This assures that every user has a consistent experience with the application.

The `weight` parameter is used to send 90% of the traffic to the main existing application, while 10% of the traffic is redirected to the experiment.

As with other examples, we'll deploy this configuration as a `ConfigMap` object in Kubernetes:

```
kubectl create configmap experiment-config --from-file=nginx.conf
```

Of course, this assumes that you have both a `web` and `experiment` service defined. If you don't, you need to create them now before you try to create the ambassador container, since `nginx` doesn't like to start if it can't find the services it is proxying to. Here are some example service configs:

```
# This is the 'experiment' service
apiVersion: v1
kind: Service
metadata:
  name: experiment
  labels:
    app: experiment
spec:
  ports:
  - port: 80
    name: web
  selector:
    # Change this selector to match your application's labels
    app: experiment
---
```

```
# This is the 'prod' service
apiVersion: v1
kind: Service
metadata:
  name: web
  labels:
    app: web
spec:
  ports:
  - port: 80
    name: web
  selector:
    # Change this selector to match your application's labels
    app: web
```

And then we will deploy `nginx` itself as the ambassador container within a pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: experiment-example
spec:
  containers:
    # This is where the application container would go, for example
    # - name: some-name
    #   image: some-image
    # This is the ambassador container
    - name: nginx
      image: nginx
      volumeMounts:
      - name: config-volume
        mountPath: /etc/nginx
  volumes:
    - name: config-volume
      configMap:
        name: experiment-config
```

You can add a second (or third, or fourth) container to the pod to take advantage of the ambassador.

# Summary

Ambassadors are a key way to simplify life for application developers. They can encapsulate complex logic that is necessary for scale or reliability, such as sharding, and provide a simplified interface which makes such a complex system easy to use. For platform engineers, ambassadors can be an important tool in constructing a powerful platform that is easy to use.

Previous chapter
< 3. The Sidecar Pattern

Next chapter
5. Adapters >